Secrets and Configuration Management

Explore how to effectively manage secrets such as API keys and passwords along with application configuration data on AWS. Learn to use AWS Secrets Manager for sensitive credential rotation, Parameter Store for configuration management, and AppConfig for controlled deployment of setting changes, ensuring secure, scalable, and auditable cloud applications.

Configuration values such as database passwords or third-party API tokens are often critical to application functionality. In cloud applications, configuration data and application secrets such as API keys, passwords, database connection strings, and third-party tokens require secure handling. Embedding these values directly into source code or deployment artifacts can lead to serious security breaches.

AWS offers services and patterns that allow us to isolate and manage secrets securely while enforcing least-privilege access and full auditability. These services include:

- Secrets Manager
- Parameter Store
- AppConfig

Before selecting a service, it's important to consider the nature of the data we're handling, whether it's personally identifiable information (PII), credentials, or general configuration settings.

Ask

Let's learn about these tools to understand how they work and when to use each.

Managing secrets with AWS Secrets Manager

AWS Secrets Manager is a fully managed AWS service designed to manage highly sensitive values like database credentials, API keys, and OAuth tokens.

Secrets are always stored encrypted using AWS KMS and retrieved via secure API calls through the AWS SDK or CLI. This means every API call is recorded by AWS CloudTrail, which allows us to see exactly who or which service accessed a specific secret and when, which is critical for security investigations and compliance.

Here is a basic Python example using Boto3 to retrieve a secret:

main.py Python

```python
import boto3
import json

def get_secret():
    client = boto3.client('secretsmanager')
    response = client.get_secret_value(SecretId='mySecretId')
    secret = json.loads(response['SecretString'])
    return secret['username'], secret['password']
```

Ask

Python code to retrieve secrets stored in AWS Secrets Manager

A key advantage of Secrets Manager is its native support for automatic secret rotation for supported services, such as Amazon RDS. This reduces the operational burden of manually rotating credentials and helps maintain compliance.

Every time a secret is updated, Secrets Manager creates a new version. To manage these versions safely, it uses staging labels. The `AWSCURRENT` label always points to the active, in-use version of the secret. During a rotation, a new version is created with the `AWSPENDING` label. Once the new secret is validated, the `AWSCURRENT` label is moved to the new version, ensuring our applications can transition to new credentials seamlessly and without downtime.

> **Protip:** Avoid storing secrets in plaintext environment variables or using broad IAM policies. Instead, use AWS Secrets Manager for automatic secret rotation and fine-grained access control, and regularly audit IAM roles to enforce least privilege.

Storing configuration settings with AWS Systems Manager Parameter Store

Parameter Store is designed for managing non-sensitive and sensitive application configuration settings. It supports both plain text and encrypted values, via KMS, using the `SecureString` type.

Ask

The service supports a path-based hierarchy, making it easy to organize parameters by environment or application. For example, we might use

`/prod/db/username` or `/dev/app/featureX` as paths.

We can also track parameter versions, retrieve them programmatically, and reference them directly from AWS services like Lambda and EC2.

Here is a simple example of retrieving a parameter in Python using Boto3:

main.py Python

```python
import boto3

def get_parameter():
    ssm = boto3.client('ssm')
    response = ssm.get_parameter(Name='/prod/db/username', WithDecryp
    return response['Parameter']['Value']
```

Retrieve parameters from AWS Parameters Store

This function retrieves an encrypted parameter named `/prod/db/username` and returns its value after decryption. This can be useful when integrating sensitive config values into our application logic.

Secrets Manager vs. Parameter Store

Although Secrets Manager and Parameter Store look similar at first glance, they cater to different needs. *Secrets Manager* is ideal for rotating sensitive credentials, while *Parameter Store* excels at managing configuration values that don't c often. Both services support encryption and version control, but only Secrets Manager provides built-in secret rotation. Parameter Store supports a hierarchical structure for organizing configurations, something Secrets Manager does not.

Ask

Centralized configuration deployment with AWS AppConfig

AWS AppConfig allows us to manage and safely deploy application configuration changes to our running applications in a controlled and monitored way. Instead of embedding configuration settings, like a new feature toggle or an external API endpoint, directly in the code, we store them centrally and securely in AppConfig. This decouples our application's configuration from its code, allowing us to make changes safely and dynamically without a full code deployment.

To understand this better, consider a simple web application for which a developer team has developed a new dark mode feature. They want to launch it, but they're cautious. What if it has a bug that only appears under heavy load? Pushing the feature to all users at once is risky. This is the perfect use case for AppConfig.

Instead of having a line in the code like `darkModeEnabled = true`, they create this configuration inside the AWS AppConfig service, which looks like follows:

index.yaml JSON

```
{
  "featureFlags": {
    "darkModeEnabled": false,
    "newCheckoutFlow": false
  },
  "loggingLevel": "INFO"
}
```

Ask

AppConfig configuration for dark mode feature

Now, the application code can easily fetch this configuration from AppConfig at runtime to decide which features to enable.

Here's a simple Python web application using the Flask framework that demonstrates how this works:

main.py Python

```
import boto3
import json
from flask import Flask

app = Flask(__name__)

# A simple cache to avoid calling AppConfig on every single request
appconfig_cache = {}

def get_app_config():
    """
    Fetches the latest configuration from AWS AppConfig.
    In a real application, you would use the AppConfig Lambda Extensi
```

Python application that use AppConf confugurations for dark mode.

This code fetches the configuration of dark mode for AppConfig and displays the frontend of the application accordingly.

For applications running on AWS Lambda, the recommended approach is the AWS AppConfig Lambda Extension. This extension runs as a separate process alongside the function, handles the session management and caching

Ask

automatically, and exposes the configuration to our function via a simple HTTP call to a local endpoint. This dramatically improves performance and reduces cost.

The most powerful feature of AppConfig is that it enables safe, gradual deployment of configuration updates without requiring code redeployments. We can configure deployments that gradually shift traffic, monitor application behavior, and roll back automatically if errors are detected. AppConfig is especially useful when managing feature flags or tuning parameters across environments at runtime.

Best practices for using secrets and configuration in applications

To build secure applications on AWS, it's necessary to handle secrets and configuration data correctly. Adhering to the following best practices helps minimize security risks, enforce governance, and improve the application's performance:

- **Never hardcode secrets:** This is the most critical rule. Secrets like database passwords, API keys, or tokens should never be stored directly in the source code, configuration files, or deployment artifacts. Always store them externally in a dedicated service like AWS Secrets Manager or Parameter Store.
- **Use IAM roles for authentication:** Whenever possible, the applications running on AWS compute services, like EC2, ECS, or Lambda, should use IAM roles to get temporary, automatically rotated credentials. Avoid using long-term IAM user access keys in your application code, as they are a significant security liability if exposed.
- **Retrieve secrets dynamically at runtime:** The application should be designed to fetch secrets and configuration values just in time when it starts

Ask

up or needs them. This ensures that the latest version is always used and that secrets are not unnecessarily persisted on disk.

- **Use environment variables for references:** It is a common but risky practice to inject secret values directly into environment variables. A better approach is to use environment variables to store the *name* or *ARN* of the secret, for example, `DB_SECRET_ARN`. The application can then use this reference to retrieve the actual secret value from Secrets Manager or Parameter Store at runtime.
- **Enforce the principle of least privilege:** The application's IAM role should only have permission to access the specific secrets and parameters it absolutely needs. Create granular IAM policies that restrict access to specific resource ARNs, for example, `arn:aws:secretsmanager:us-east-1:123456789012:secret:my-app/db-secret-AbCdEf`. We can further scope down permissions using condition keys, such as restricting access based on resource tags using `ssm:ResourceTag`.
- **Secure the network with VPC endpoints:** For applications running within a private VPC, use interface VPC endpoints for Secrets Manager, Parameter Store, and AppConfig. This allows the application to access these services over the private AWS network, ensuring that sensitive configuration data never traverses the public internet.
- **Audit everything with AWS CloudTrail:** Ensure CloudTrail is enabled in the account. It automatically logs every single API call made to these services, including `GetSecretValue` and `GetParameter`. This provides an immutable audit trail that is essential for security investigations and compliance, allowing you to see exactly who or what accessed the secrets and when.

Integrating secrets into CI/CD workflows

Ask

Secrets and configuration values are often required during the build and deployment phases. AWS CodeBuild and CodePipeline allow us to securely inject

these values as environment variables, referencing them from Secrets Manager or Parameter Store.

For example, in a CodeBuild file `buildspec.yml`, we can reference a secure parameter as follows:

```yaml
environment_variables:
  plaintext:
    STAGE: "dev"
  parameter_store:
    DB_PASSWORD: "/dev/db/password"
```

Use secure parameters in CI/CD workflows

This will retrieve the value stored in Parameter Store at runtime and make it available to the build container as `DB_PASSWORD`.

In a CodePipeline definition using CloudFormation or the console, we can specify secrets to be injected during the CodeBuild phase by linking to Secrets Manager ARNs in the environment settings.

Additionally, AppConfig can be used to manage configuration values tied to stages in our pipeline, such as turning on a feature flag in staging before promoting it to production.

Conclusion

Secrets and configuration management are critical for developing secure applications in AWS. Use AWS Secrets Manager for sensitive credentials that require rotation, Parameter Store for structured configuration, and AppConfig for safe rollout of runtime settings. Apply the principle of least privilege, avoid hardcoding secrets, and use audit tools like CloudTrail to track access and changes.

Ask